B.Tech. · Semester V · UE24CS320A

END SEMESTER ASSESSMENT

Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Storage Maintenance Under Concurrent Analytical Workloads

Project ID PW_25_PKA_02
Guide Dr. Prafullata Kiran Auradkar

Shashank Keshava Murthy · PES1UG24CS434
Shashank D · PES1UG24CS433
Prachi Jalan · PES1UG24CS326
Samarth Rajesh Rao · PES1UG24AM244

Outline

Problem, implementation and evidence

Why compaction timing is worth studying; what Phase 1 aims to establish.

Architecture, datasets and the responsibilities of the A/B/C components.

Actual execution screenshots, historical results and what they do not prove.

Review feedback addressed, remaining limitations and next work.

Abstract

Choose maintenance time using workload and queue information

Compaction combines small files, but it competes with analytical queries for resources.

We built a remote Spark/Iceberg testbed with trace-driven requests, observation and audit tools.

We compare learned estimators with simple baselines before claiming that a learned timing policy is useful.

Current outcome: implementation and diagnostic progress; no accepted SF10 learned scheduling advantage.

SHASHANK KESHAVA MURTHY · SHASHANK D · PRACHI JALAN · SAMARTH RAJESH RAO

Suggestions from Review 1 and Review 2

Make the work visible and explain the dataset

Panel feedback	What we prepared
Review 1: explain TPC-H structure	Eight tables, column counts, verified SF10 rows and a clear Snowset/SQL mapping.
Review 2: show actual screenshots and what has started	Labelled process, Spark UI, dashboard and audit screenshots with an explanation beside each.

The evidence demonstrates that the project has started; it does not turn an incomplete experiment into a successful result.

Understanding of the Problem

Response time includes waiting, not just query execution

Why maintain?

Too many files can add planning and file-access work. Compaction changes the layout.

Why choose a time?

The rewrite may compete with queries. A low-load opportunity may be better than a busy period.

What must improve?

Arrival-to-result response, with completed work and remaining maintenance accounted for.

SHASHANK KESHAVA MURTHY · SHASHANK D · PRACHI JALAN · SAMARTH RAJESH RAO

Objectives and Scope

Test the timing hypothesis before adding model complexity

Build a reproducible Spark-on-Iceberg execution and measurement foundation.

Compare simple medians, shared estimators and separate A/B latency candidates on whole workload families.

Use C to forecast offered arrivals from past observations; measure its value and overhead.

Evaluate start-now, wait or defer choices only after fresh physical evidence passes its gates.

SHASHANK KESHAVA MURTHY · SHASHANK D · PRACHI JALAN · SAMARTH RAJESH RAO

Dataset Description

TPC-H supplies the data and SQL; Snowset supplies arrival timing

TPC-H

Eight related tables and 61 columns. Our verified SF10 lineitem table contains 59,986,052 rows.

Snowset

Recorded arrival timestamps set request spacing. The original Snowflake SQL and schema are not replayed.

Controlled workload

Six TPC-H-derived SQL templates are assigned to arrivals. This is not the full official 22-query benchmark.

TPC-H Structure

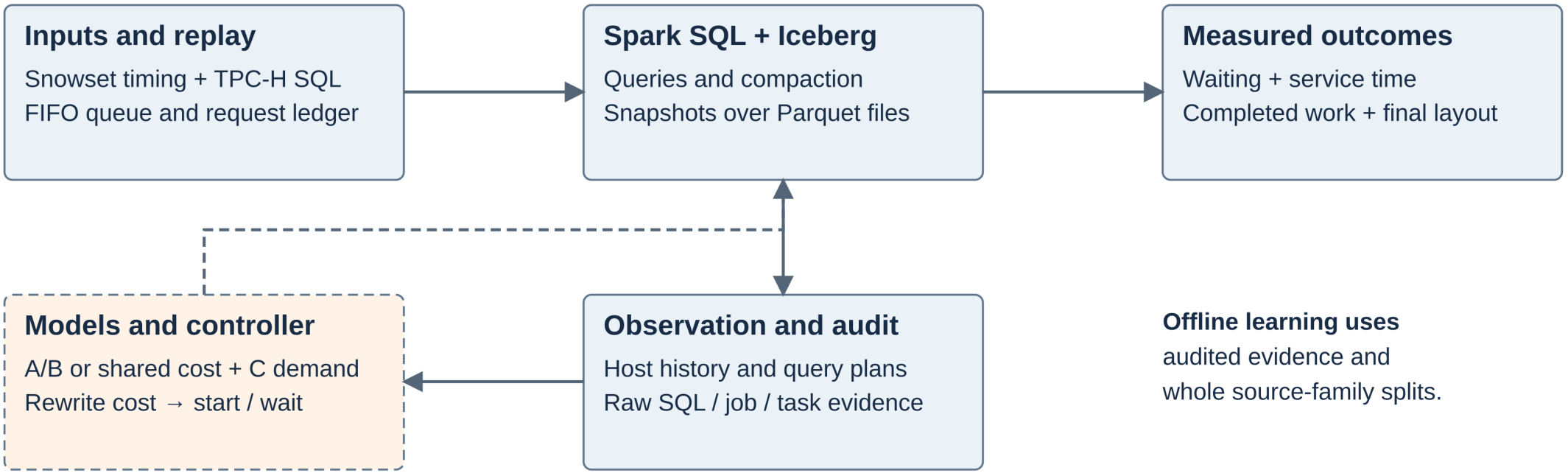
Eight linked tables form a wholesale business dataset

Table	Columns	SF10 rows
region / nation	3 / 4	5 / 25
supplier / customer	7 / 8	100,000 / 1,500,000
part / partsupp	9 / 5	2,000,000 / 8,000,000
orders / lineitem	9 / 16	15,000,000 / 59,986,052

SHASHANK KESHAVA MURTHY · SHASHANK D · PRACHI JALAN · SAMARTH RAJESH RAO

Proposed Architecture

The execution path is separate from the learned decision loop



Solid: execution and evidence foundation. Dashed: proposed learned decision loop; benefit and calibration remain unproven.

Model Responsibilities

A/B estimate service time; C estimates future demand

Component	Output	Important boundary
A	Service time with request pending	Pending ≠ measured causal interference
B	Service time without request pending	Conditioned on actual physical layout
Shared comparator	Service time + state as an input	Same evidence splits and predictor budget
C	Future offered arrival count	Counts ≠ CPU load; past-only inputs
Rewrite estimate	Duration and layout transition	Separate measured target

Available Inputs

22 registered SF10 candidate inputs, all available before a decision

Query and layout

Query identity; file count, row count and bytes; eight query-plan operator counts.

Queue and state

Current queue depth and whether the maintenance client request is pending.

Host history

CPU, memory, read and write rates for both 60-second and 300-second windows.

Missing or unavailable history is not filled in. Post-query outcomes do not become predictors.

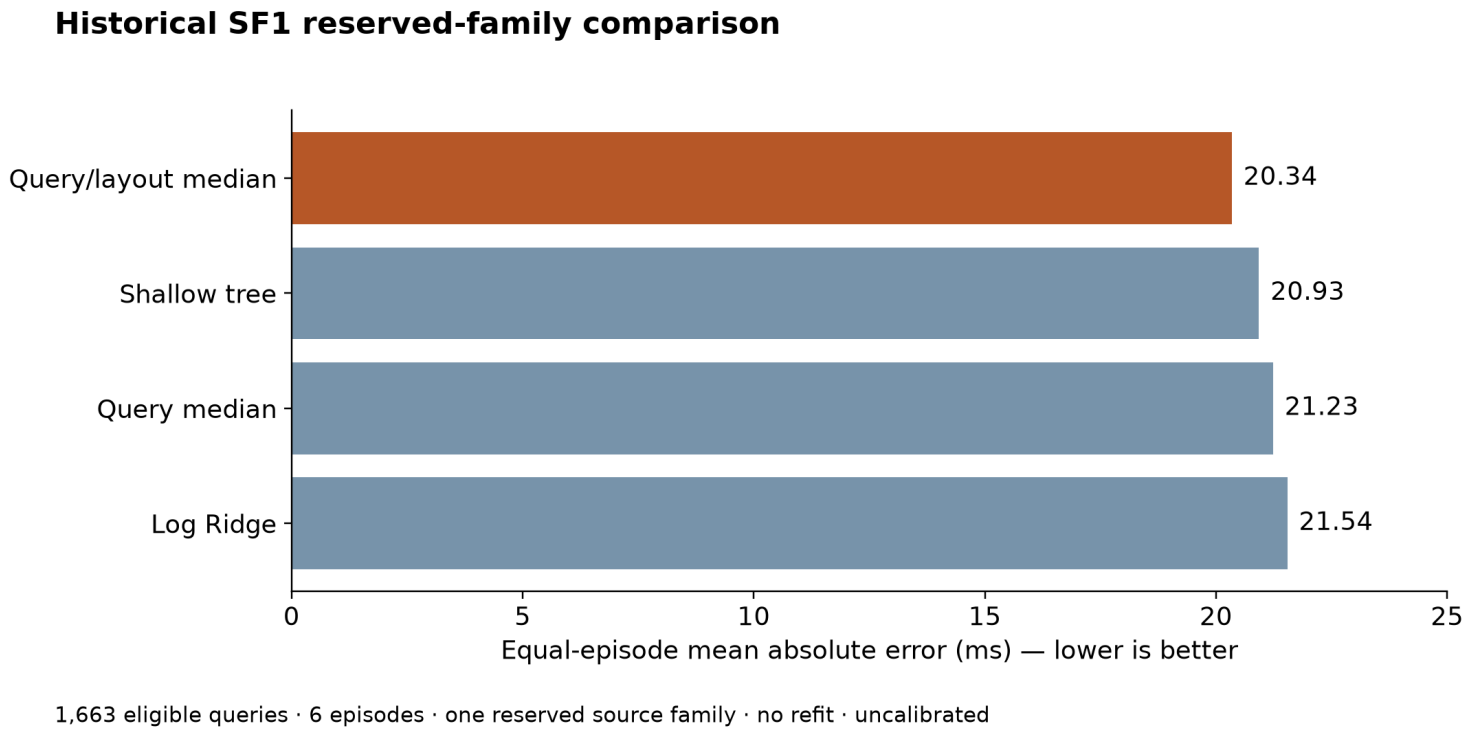
Implementation Progress

A working foundation with explicit acceptance boundaries

Built or observed	Current boundary
Remote Spark/Iceberg, replay and queue ledger	Real execution and recorded offered/completed/queued work
Observation, archive, receipts and hashes	Full SF10 acceptance remains blocked
Exploratory A/B/shared models and medians	SF1 development artifacts; no calibrated scheduler
SF10 ingestion and separate compaction capability	64 → 4 files in a capability run; not a policy win

Historical SF1 Results

The strongest simple baseline beat the learned query models



What this tells us

18 historical SF1 episodes; 9,923 query labels across three families.

Reserved test: 1,663 eligible calls. These are prediction errors, not scheduling benefit.

Exploratory Model Training

The A/B pipeline trained and reloaded real candidate artifacts

What ran

Raw verification → frozen specification → fit → artifact reload audit. Ridge and shallow-tree bundles were saved.

What data supported it

335 complete-history SF1 calls: 16 pending-maintenance A and 319 nonpending B. One development family.

What remains open

Different layouts confound the states. This fit does not establish held-out accuracy, uncertainty or causal slowdown.

How to read the screenshots

Eight original images show four views captured twice on November 11.

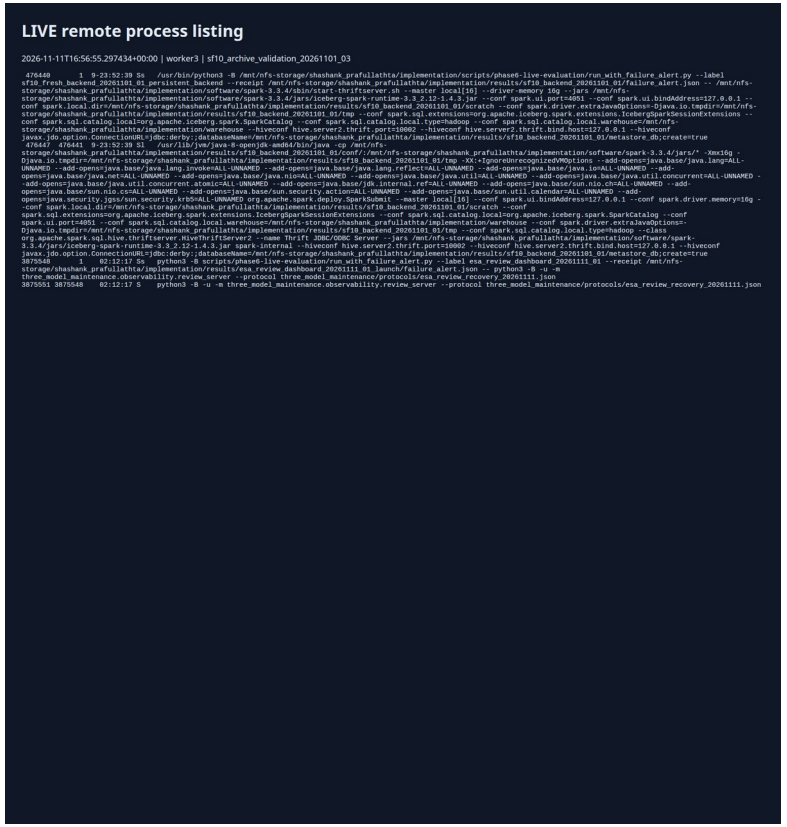
A live process listing proves a service was running at capture time. Spark was idle during these captures.

The dashboard and audit show the historical failed SF10 control; they are not accepted training evidence.

Each following view has a labelled explanation. Full originals and exact crops accompany the deck.

Recorded Execution · B01

Remote Spark and monitoring processes



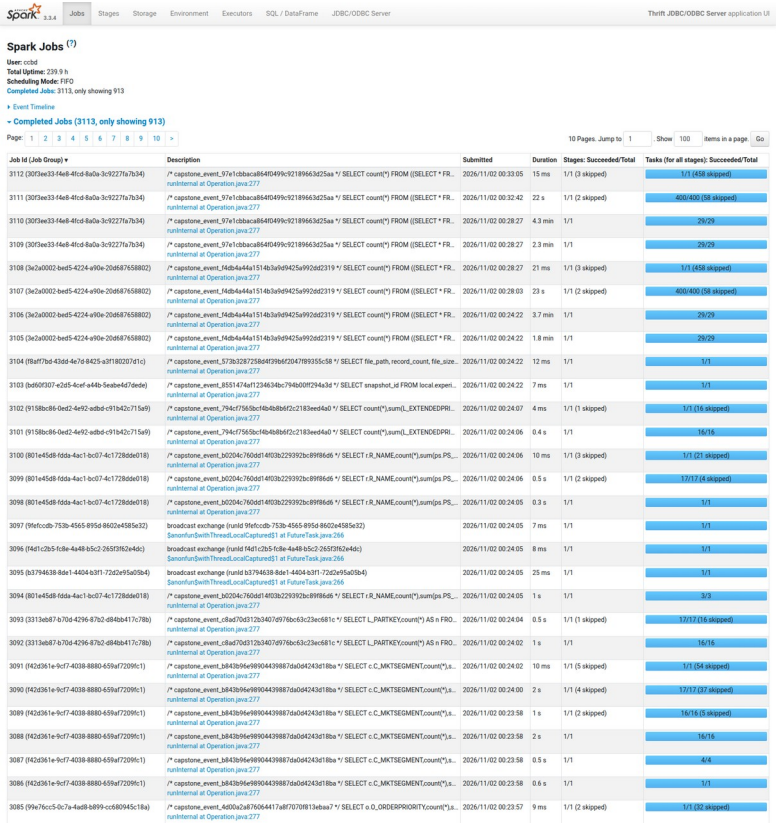
11 November 2026, 22:26 IST

The process list shows the Spark launcher, Java execution process and our read-only review server on worker3. It establishes that the remote software stack was running when captured. It does not show an active training or compaction experiment.

Historical capture. No experiment was running.

Recorded Execution · B02

Spark UI and completed execution history



11 November 2026, 22:26 IST

The Spark UI lists completed jobs with SQL descriptions, submission times, durations and successful task counts. The application reports 3,113 completed jobs and only 913 displayed in its retained view. These are application-level totals, not counts of independent experiments or accepted training examples.

Historical capture. No experiment was running.

SHASHANK KESHAVA MURTHY · SHASHANK D · PRACHI JALAN · SAMARTH RAJESH RAO

PES UNIVERSITY

Recorded Execution · B03

Recorded SF10 workload and table state



11 November 2026, 22:26 IST

The dashboard combines a live service observation with explicitly historical experiment data. The failed control recorded 463 completed calls and 2,668 queued requests. Initial and final table state remained at 64 files and 59,986,052 rows. The graph shows the last 240 recorded service times; it is not a policy comparison.

Historical capture. No experiment was running.

Recorded Execution · B04

Preserved failed SF10 campaign



11 November 2026, 22:26 IST

The campaign ledger records zero passed episodes, one failed episode and one unattempted maintenance episode. Model fitting and scheduler authorization are false. A later ownership diagnostic identified the description-based attribution defect, but it did not retroactively pass this run.

Historical capture. No experiment was running.

Screenshot Detail

The runtime stack is visible in the process list



Python wrapper launches the backend.

Java runs Spark with Iceberg support.

The review server reads and displays evidence.

This confirms the execution environment was running. It does not show an active model fit.

SF10 Compaction Capability

A separate historical run changed 64 files into 4

64 files → **4 files**

1,714,218,564 bytes reported rewritten in the saved procedure result.

This demonstrates actual layout rewriting, not improved query latency or better maintenance timing.

In the later failed control, no compaction ran and the layout correctly remained at 64 files.

What Failed and What We Learned

The audit rejected generated jobs without the SQL marker

Original outcome

0 passed / 1 failed / 1 unattempted. The maintenance arm was not launched. No SF10 fit followed.

Diagnostic progress

463 exact SQL matches. All 308 rejected job links belonged to those marked SQL executions. No unresolved links.

Required repair

Capture direct SQL → job → stage → task ownership prospectively, while still rejecting unknown work.

The old run remains failed. Ownership diagnosis does not prove complete task evidence.

Applicability in Different Domains

Three potential settings with the same maintenance trade-off

Retail analytics

Order and sales reports share the system with file maintenance. Bursty reporting demand creates timing choices.

Financial reporting

Reconciliation and analytical queries can overlap with maintenance windows. Queue response and completion matter.

Industrial analytics

Sensor history accumulates in lakehouse tables. File consolidation may compete with interactive analysis.

These are proposed uses. Our measurements are from the controlled Spark/Iceberg benchmark.

Related Work

The comparison must be against strong existing approaches

Work	Relevant idea	Our remaining question
AutoComp	Automated table compaction	Queue-aware timing needs direct evaluation
SILK	Manage maintenance interference	Different LSM / resource-scheduler setting
Learned cost model	Plan-aware prediction for SQL routing	Prediction does not prove timing benefit
Smart Compaction	Metadata-based compaction utility	Metadata ML alone is not novelty

Phase 1 Conclusion

We have started the implementation and can show its evidence

Built: remote execution, dataset preparation, replay, observation, candidate training and monitoring.

Observed: historical SF1 comparison and separate SF10 compaction capability.

Diagnosed: a concrete SQL-job attribution defect in the failed SF10 campaign.

Still pending: fresh accepted SF10 pair, independent calibration and a demonstrated scheduling advantage.

Next Work

Repair, validate, then compare on fresh evidence

1 Attribution repair

Record direct SQL/job membership and complete task evidence. Validate without weakening the foreign-work gate.

2 Fresh matched comparison

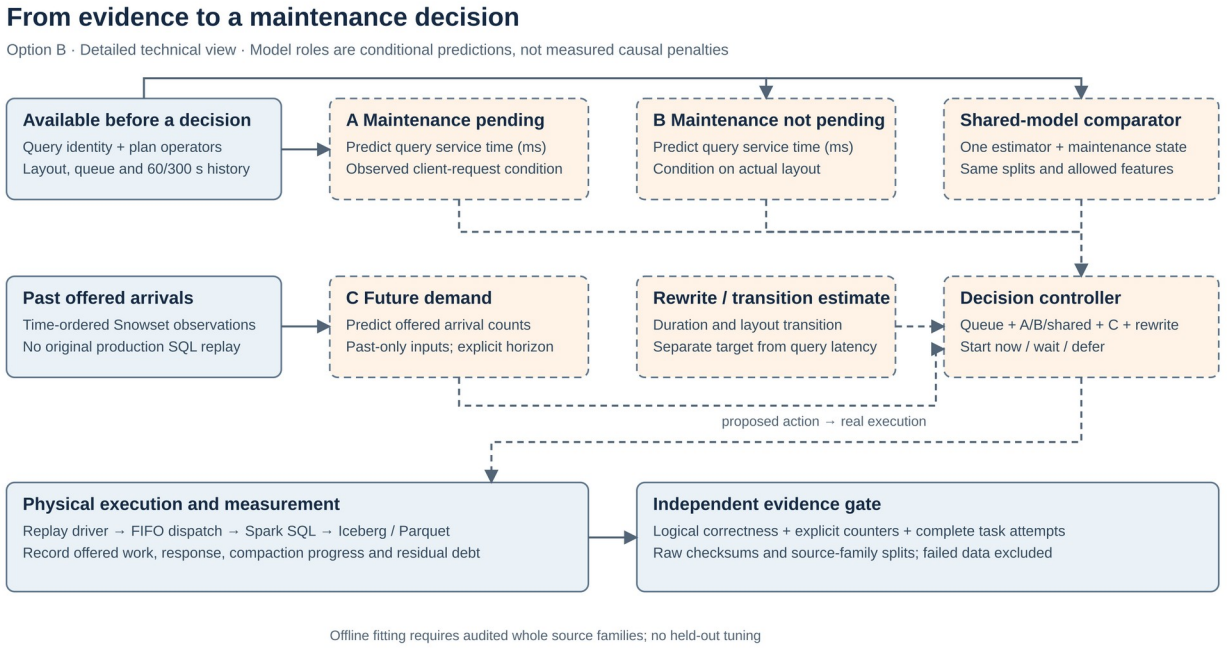
Register new source windows and tables. Run control and compaction with equivalent workload accounting.

3 Model and policy evaluation

Compare medians, shared and separate estimators; test forecasting and total online overhead.

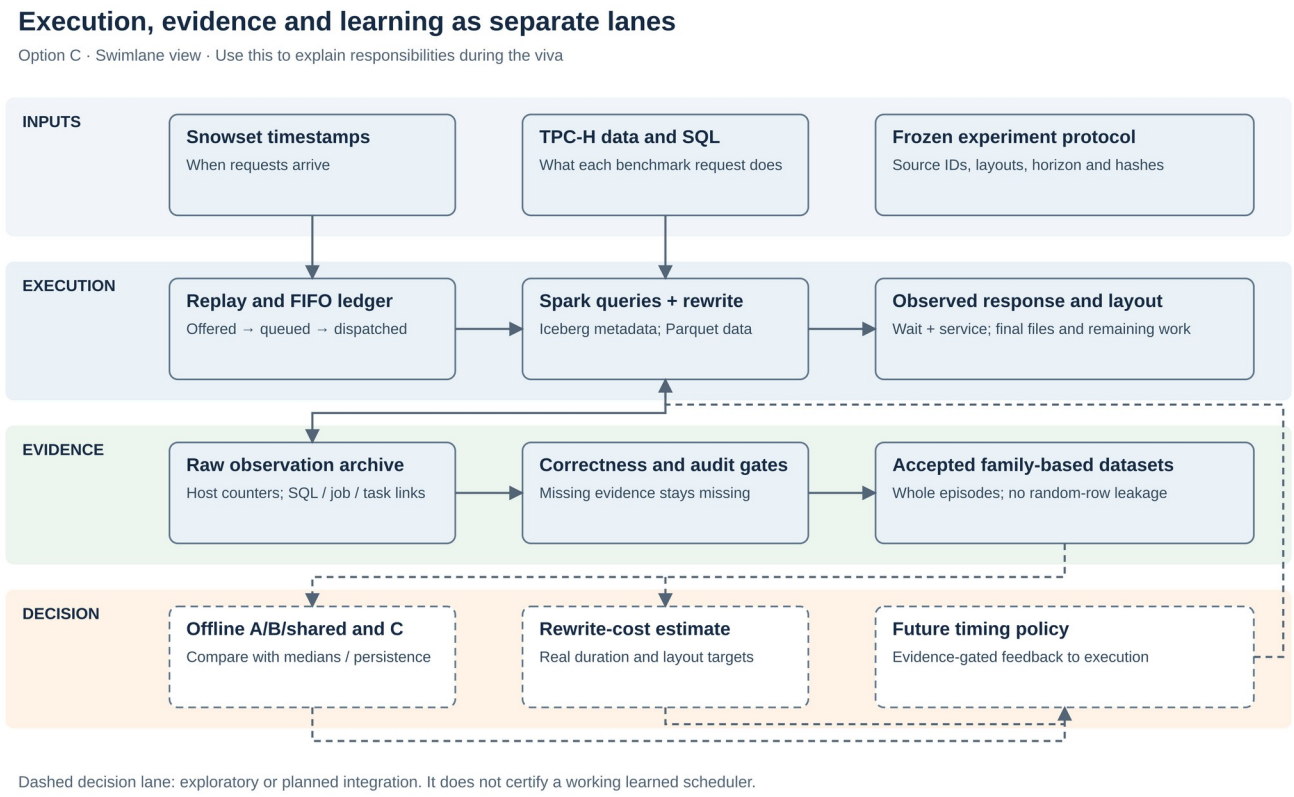
Architecture Option B

Detailed model and evidence contract



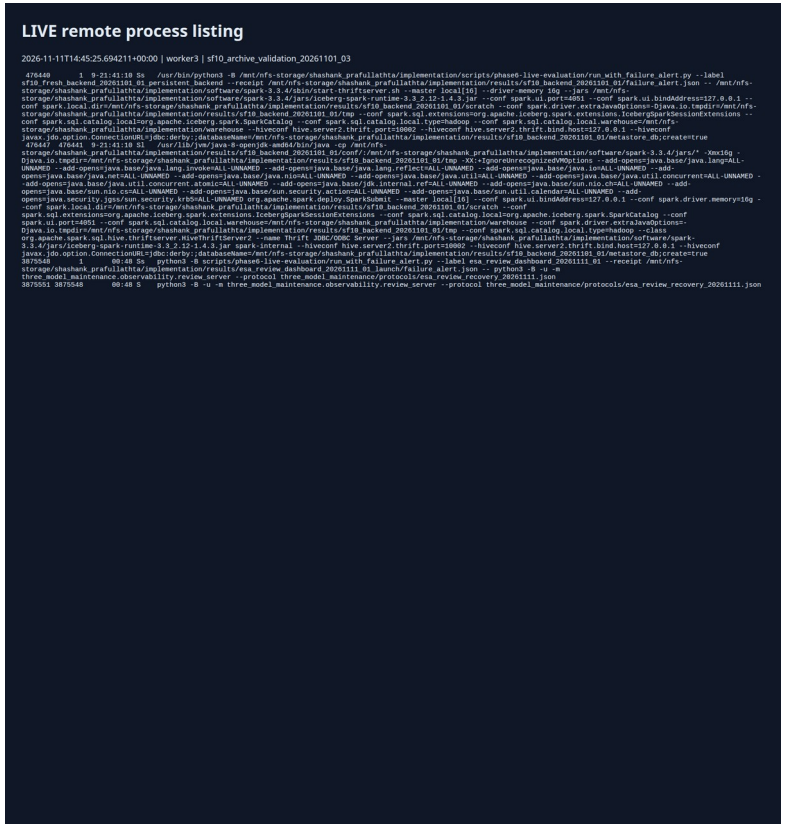
A − B is not a causal slowdown estimate. No calibrated interval or scheduler promotion without independent accepted blocks.

Explain the system as four responsibility lanes



Earlier Capture · A01

Remote Spark and monitoring processes



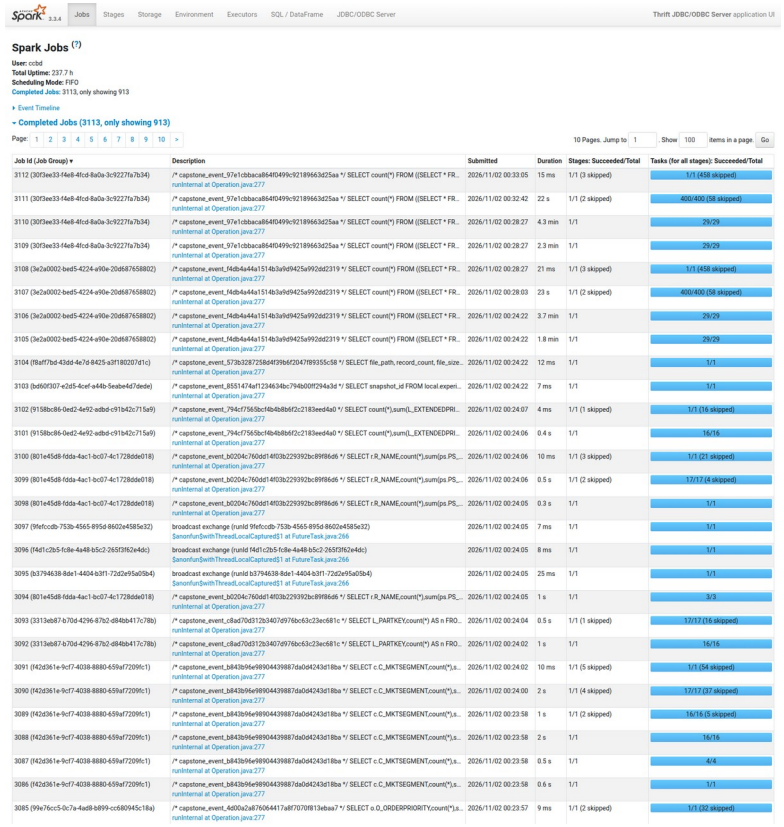
11 November 2026, 20:15 IST

The process list shows the Spark launcher, Java execution process and our read-only review server on worker3. It establishes that the remote software stack was running when captured. It does not show an active training or compaction experiment.

Earlier observation of the same system; not another experiment.

Earlier Capture · A02

Spark UI and completed execution history



11 November 2026, 20:15 IST

The Spark UI lists completed jobs with SQL descriptions, submission times, durations and successful task counts. The application reports 3,113 completed jobs and only 913 displayed in its retained view. These are application-level totals, not counts of independent experiments or accepted training examples.

Earlier observation of the same system; not another experiment.

Earlier Capture · A03

Recorded SF10 workload and table state



11 November 2026, 20:15 IST

The dashboard combines a live service observation with explicitly historical experiment data. The failed control recorded 463 completed calls and 2,668 queued requests. Initial and final table state remained at 64 files and 59,986,052 rows. The graph shows the last 240 recorded service times; it is not a policy comparison.

Earlier observation of the same system; not another experiment.

Earlier Capture · A04

Preserved failed SF10 campaign



11 November 2026, 20:15 IST

The campaign ledger records zero passed episodes, one failed episode and one unattempted maintenance episode. Model fitting and scheduler authorization are false. A later ownership diagnostic identified the description-based attribution defect, but it did not retroactively pass this run.

Earlier observation of the same system; not another experiment.

References and Evidence

Primary references and saved project records

TPC-H specification; Snowset source documentation; Spark 3.3.4 and Iceberg 1.4.3 documentation.

AutoComp (2025); SILK (2019); Strausz et al. cross-engine cost model (2025); Smart Compaction preprint (2026).

Historical reserved evaluation: consistent_reserved_evaluation_20261017.

SF10 capability, failed archive03, SQL ownership diagnostic and original screenshot manifest.

Thank You

Questions and demonstration

Work completed, evidence retained, limitations stated.

The report and screenshot index provide the source for every reported result.